

Logistic Regression Remains Competitive for Diabetes Prediction in Small Clinical Datasets

Autumn Park

2026-05-06

Introduction: This study systematically compares the performance of logistic regression with contemporary machine learning algorithms for diabetes prediction. Compare and select the best model for predicting diabetes. while providing a comprehensive evaluation of the methodological complexities, inherent limitations, and empirical results associated with logistic regression and alternative approaches.

Objective: The purpose of the study is to evaluate whether logistic regression is comparable with more modern machine learning algorithms for diabetes prediction. This study emphasizes productive predictive performance calibration interpretation.

1. Load data

```
data("PimaIndiansDiabetes2", package = "mlbench")
df <- PimaIndiansDiabetes2
```

2. Clean data

```
df <- df %>%
  mutate(diabetes = factor(diabetes, levels = c("neg", "pos")))

missing_summary <- sapply(df, function(x) sum(is.na(x)))
#print(missing_summary)

# Median imputation for numeric predictors
preprocess_obj <- preProcess(df[, names(df) != "diabetes"], method = c("medianImpute"))
x_imputed <- predict(preprocess_obj, df[, names(df) != "diabetes"])

df_clean <- bind_cols(x_imputed, diabetes = df$diabetes)

#No missing values
colSums(is.na(df_clean))
```

```
## pregnant glucose pressure triceps insulin mass pedigree age
##          0           0           0           0           0           0           0
## diabetes
##          0
```

3. Train-test split / Cross-validation / Standardization

```
train_index <- createDataPartition(df_clean$diabetes, p = 0.80, list = FALSE)
train_data <- df_clean[train_index, ]
test_data <- df_clean[-train_index, ]

train_control <- trainControl(
  method = "repeatedcv",
  number = 10,
  repeats = 3,
  classProbs = TRUE,
  summaryFunction = twoClassSummary
)
```

4. Standardization for models that benefit from it

```
x_train <- train_data %>% select(-diabetes) %>% as.matrix()
x_test <- test_data %>% select(-diabetes) %>% as.matrix()

pp_scale <- preProcess(train_data %>% select(-diabetes), method = c("center", "scale"))
x_train_scaled <- predict(pp_scale, train_data %>% select(-diabetes)) %>% as.matrix()
x_test_scaled <- predict(pp_scale, test_data %>% select(-diabetes)) %>% as.matrix()

y_train <- train_data$diabetes
y_test <- test_data$diabetes

# Numeric outcome for glmnet
y_train_num <- ifelse(y_train == "pos", 1, 0)
y_test_num <- ifelse(y_test == "pos", 1, 0)
```

Models

5. Logistic regression

```
log_model <- glm(diabetes ~ ., data = train_data, family = binomial())

summary(log_model)

##
## Call:
## glm(formula = diabetes ~ ., family = binomial(), data = train_data)
```

```
##
## Coefficients:
##           Estimate Std. Error z value Pr(>|z|)
## (Intercept) -8.941792   0.898639  -9.950  < 2e-16 ***
## pregnant    0.106776   0.036415   2.932  0.00337 **
## glucose     0.039870   0.004483   8.894  < 2e-16 ***
## pressure   -0.012891   0.010188  -1.265  0.20579
## triceps    -0.003454   0.014828  -0.233  0.81582
## insulin    -0.002915   0.001328  -2.196  0.02809 *
## mass       0.100930   0.020417   4.943  7.68e-07 ***
## pedigree   0.978949   0.331924   2.949  0.00318 **
## age        0.016441   0.010772   1.526  0.12695
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 796.05  on 614  degrees of freedom
## Residual deviance: 571.47  on 606  degrees of freedom
## AIC: 589.47
##
## Number of Fisher Scoring iterations: 5
```

```
# Predicted probabilities
log_prob <- predict(log_model, newdata = test_data, type = "response")
log_pred <- ifelse(log_prob >= 0.5, "pos", "neg") %>% factor(levels = c("neg", "pos"))
```

6. Penalized logistic regression (LASSO)

```
cv_lasso <- cv.glmnet(
  x = x_train_scaled,
  y = y_train_num,
  family = "binomial",
  alpha = 1,
  nfolds = 10
)

lasso_prob <- predict(cv_lasso, newx = x_test_scaled, s = "lambda.min", type = "response")[, 1]
lasso_pred <- ifelse(lasso_prob >= 0.5, "pos", "neg") %>% factor(levels = c("neg", "pos"))
```

7. Random forest

```
rf_model <- ranger(
  diabetes ~ .,
  data = train_data,
  probability = TRUE,
  num.trees = 500,
  importance = "impurity"
)
```

```
rf_prob <- predict(rf_model, data = test_data)$predictions[, "pos"]
rf_pred <- ifelse(rf_prob >= 0.5, "pos", "neg") %>% factor(levels = c("neg", "pos"))
```

8. XGBoost

```
# XGBoost requires a numeric matrix and numeric 0/1 outcome.
# The positive diabetes class is coded as 1.
dtrain <- xgb.DMatrix(data = x_train, label = y_train_num)
dtest  <- xgb.DMatrix(data = x_test, label = y_test_num)

xgb_params <- list(
  objective = "binary:logistic",
  eval_metric = "auc",
  max_depth = 3,
  eta = 0.05,
  subsample = 0.80,
  colsample_bytree = 0.80
)

set.seed(1234)
xgb_model <- xgb.train(
  params = xgb_params,
  data = dtrain,
  nrounds = 150,
  verbose = 0
)

xgb_prob <- predict(xgb_model, newdata = dtest)
xgb_pred <- ifelse(xgb_prob >= 0.5, "pos", "neg") %>%
  factor(levels = c("neg", "pos"))
```

9. Evaluation function

```
evaluate_model <- function(truth, prob, pred, model_name) {
  truth <- factor(truth, levels = c("neg", "pos"))
  pred <- factor(pred, levels = c("neg", "pos"))
  prob <- as.numeric(prob)

  cm <- confusionMatrix(pred, truth, positive = "pos")
  roc_obj <- pROC::roc(truth, prob, levels = c("neg", "pos"), direction = "<")

  tibble(
    Model = model_name,
    Accuracy = as.numeric(cm$overall["Accuracy"]),
    Sensitivity = as.numeric(cm$byClass["Sensitivity"]),
    Specificity = as.numeric(cm$byClass["Specificity"]),
    Precision = as.numeric(cm$byClass["Pos Pred Value"]),
    Recall = as.numeric(cm$byClass["Sensitivity"]),
    F1 = 2 * (
```

```

    as.numeric(cm$byClass["Pos Pred Value"]) * as.numeric(cm$byClass["Sensitivity"])
  ) / (
    as.numeric(cm$byClass["Pos Pred Value"]) + as.numeric(cm$byClass["Sensitivity"])
  ),
  AUC = as.numeric(pROC::auc(roc_obj))
)
}

results_log <- evaluate_model(y_test, log_prob, log_pred, "Logistic Regression")
results_lasso <- evaluate_model(y_test, lasso_prob, lasso_pred, "LASSO Logistic")
results_rf <- evaluate_model(y_test, rf_prob, rf_pred, "Random Forest")
results_xgb <- evaluate_model(y_test, xgb_prob, xgb_pred, "XGBoost")

results_all <- bind_rows(results_log, results_lasso, results_rf, results_xgb)
#print(results_all)

```

10. ROC curves

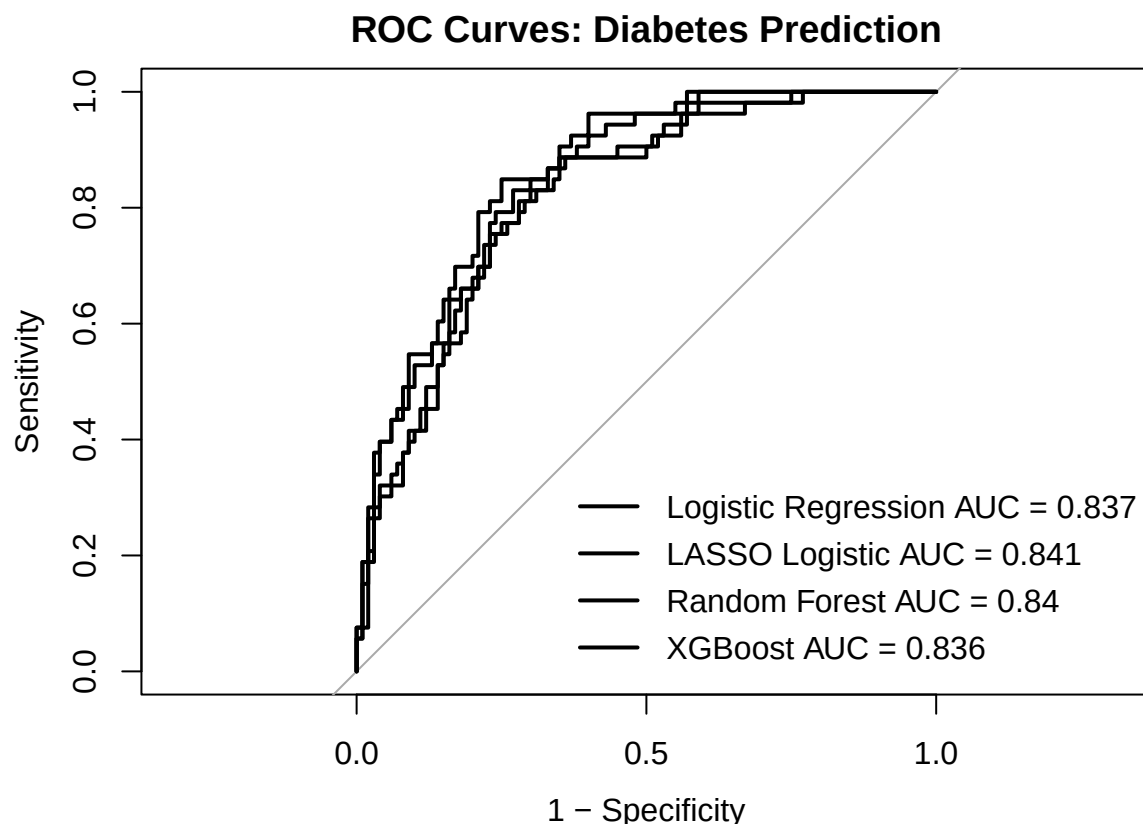
```

roc_log <- pROC::roc(y_test, log_prob, levels = c("neg", "pos"), direction = "<")
roc_lasso <- pROC::roc(y_test, lasso_prob, levels = c("neg", "pos"), direction = "<")
roc_rf <- pROC::roc(y_test, rf_prob, levels = c("neg", "pos"), direction = "<")
roc_xgb <- pROC::roc(y_test, xgb_prob, levels = c("neg", "pos"), direction = "<")

plot(roc_log, main = "ROC Curves: Diabetes Prediction", legacy.axes = TRUE)
plot(roc_lasso, add = TRUE)
plot(roc_rf, add = TRUE)
plot(roc_xgb, add = TRUE)

legend(
  "bottomright",
  legend = c(
    paste0("Logistic Regression AUC = ", round(pROC::auc(roc_log), 3)),
    paste0("LASSO Logistic AUC = ", round(pROC::auc(roc_lasso), 3)),
    paste0("Random Forest AUC = ", round(pROC::auc(roc_rf), 3)),
    paste0("XGBoost AUC = ", round(pROC::auc(roc_xgb), 3))
  ),
  lwd = 2,
  bty = "n"
)

```



11. Model calibration

Calibration evaluates whether predicted probabilities correspond to observed outcome rates. This is especially important in clinical prediction, where a probability should be meaningful for risk communication and decision-making.

```
calibration_df <- tibble(
  truth = ifelse(y_test == "pos", 1, 0),
  Logistic = log_prob,
  LASSO = lasso_prob,
  `Random Forest` = rf_prob,
  XGBoost = xgb_prob
)

# Function for binned calibration/reliability curves
calibrate_model <- function(prob, truth, bins = 10) {
  tibble(prob = prob, truth = truth) %>%
    mutate(bin = dplyr::ntile(prob, bins)) %>%
    group_by(bin) %>%
    summarise(
      mean_predicted_probability = mean(prob, na.rm = TRUE),
      observed_event_rate = mean(truth, na.rm = TRUE),
      n = n(),

```

```

    .groups = "drop"
  )
}

calibration_all <- bind_rows(
  calibrate_model(calibration_df$Logistic, calibration_df$truth) %>%
    mutate(model = "Logistic Regression"),
  calibrate_model(calibration_df$LASSO, calibration_df$truth) %>%
    mutate(model = "LASSO Logistic Regression"),
  calibrate_model(calibration_df$`Random Forest`, calibration_df$truth) %>%
    mutate(model = "Random Forest"),
  calibrate_model(calibration_df$XGBoost, calibration_df$truth) %>%
    mutate(model = "XGBoost")
)

print(calibration_all)

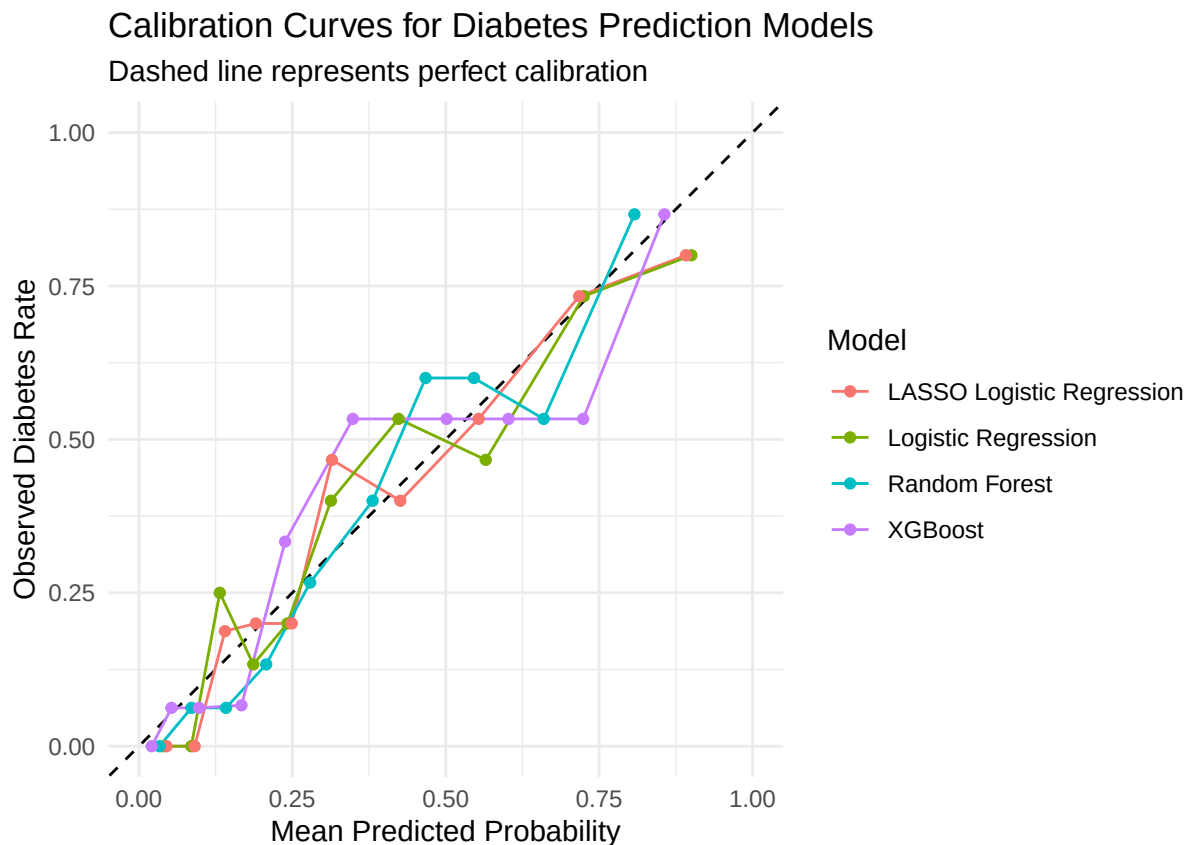
## # A tibble: 40 x 5
##   bin mean_predicted_probability observed_event_rate     n model
##   <int>                <dbl>          <dbl> <int> <chr>
## 1     1                0.0404            0     16 Logistic Regressi-
## 2     2                0.0854            0     16 Logistic Regressi-
## 3     3                0.132             0.25    16 Logistic Regressi-
## 4     4                0.186             0.133   15 Logistic Regressi-
## 5     5                0.242             0.2     15 Logistic Regressi-
## 6     6                0.313             0.4     15 Logistic Regressi-
## 7     7                0.423             0.533   15 Logistic Regressi-
## 8     8                0.565             0.467   15 Logistic Regressi-
## 9     9                0.725             0.733   15 Logistic Regressi-
## 10    10                0.900             0.8     15 Logistic Regressi-
## # i 30 more rows

```

```

ggplot(
  calibration_all,
  aes(
    x = mean_predicted_probability,
    y = observed_event_rate,
    color = model
  )
) +
  geom_abline(intercept = 0, slope = 1, linetype = "dashed") +
  geom_line() +
  geom_point() +
  coord_equal(xlim = c(0, 1), ylim = c(0, 1)) +
  labs(
    title = "Calibration Curves for Diabetes Prediction Models",
    subtitle = "Dashed line represents perfect calibration",
    x = "Mean Predicted Probability",
    y = "Observed Diabetes Rate",
    color = "Model"
  ) +
  theme_minimal()

```



Brier score

The Brier score measures the mean squared difference between predicted probabilities and observed outcomes. Lower values indicate better overall probability accuracy.

```
brier_score <- function(truth, prob) {
  mean((truth - prob)^2, na.rm = TRUE)
}

brier_tbl <- tibble(
  model = c(
    "Logistic Regression",
    "LASSO Logistic Regression",
    "Random Forest",
    "XGBoost"
  ),
  brier_score = c(
    brier_score(calibration_df$truth, calibration_df$Logistic),
    brier_score(calibration_df$truth, calibration_df$LASSO),
    brier_score(calibration_df$truth, calibration_df$Random Forest),
    brier_score(calibration_df$truth, calibration_df$XGBoost)
  )
)
```



```
) %>%
  arrange(brier_score)

print(brier_tbl)
```

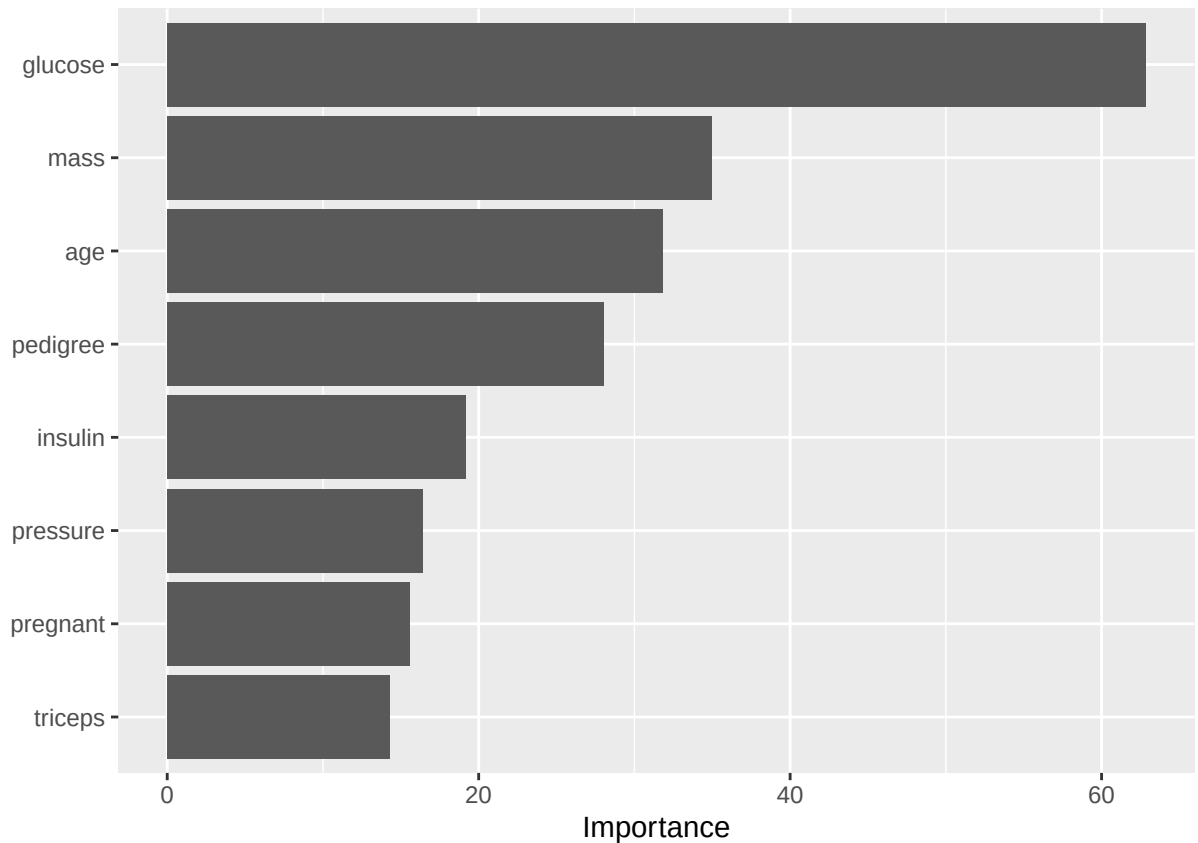
```
## # A tibble: 4 x 2
##   model                brier_score
##   <chr>                <dbl>
## 1 Random Forest        0.154
## 2 LASSO Logistic Regression 0.155
## 3 Logistic Regression    0.156
## 4 XGBoost              0.158
```

12. Odds ratios for logistic regression

```
## # A tibble: 9 x 7
##   term      estimate std.error statistic  p.value  conf.low conf.high
##   <chr>      <dbl>    <dbl>    <dbl>    <dbl>    <dbl>    <dbl>
## 1 pedigree  2.66      0.332     2.95 3.18e- 3 1.40      5.14
## 2 pregnant  1.11      0.0364    2.93 3.37e- 3 1.04      1.20
## 3 mass      1.11      0.0204    4.94 7.68e- 7 1.06      1.15
## 4 glucose   1.04      0.00448   8.89 5.89e-19 1.03      1.05
## 5 age       1.02      0.0108    1.53 1.27e- 1 0.995     1.04
## 6 insulin   0.997     0.00133   -2.20 2.81e- 2 0.994     1.000
## 7 triceps   0.997     0.0148   -0.233 8.16e- 1 0.968     1.03
## 8 pressure  0.987     0.0102   -1.27 2.06e- 1 0.968     1.01
## 9 (Intercept) 0.000131 0.899    -9.95 2.51e-23 0.0000211 0.000718
```

13. Variable importance from random forest

```
vip(rf_model, num_features = 8)
```



14. Variable importance from XGBoost

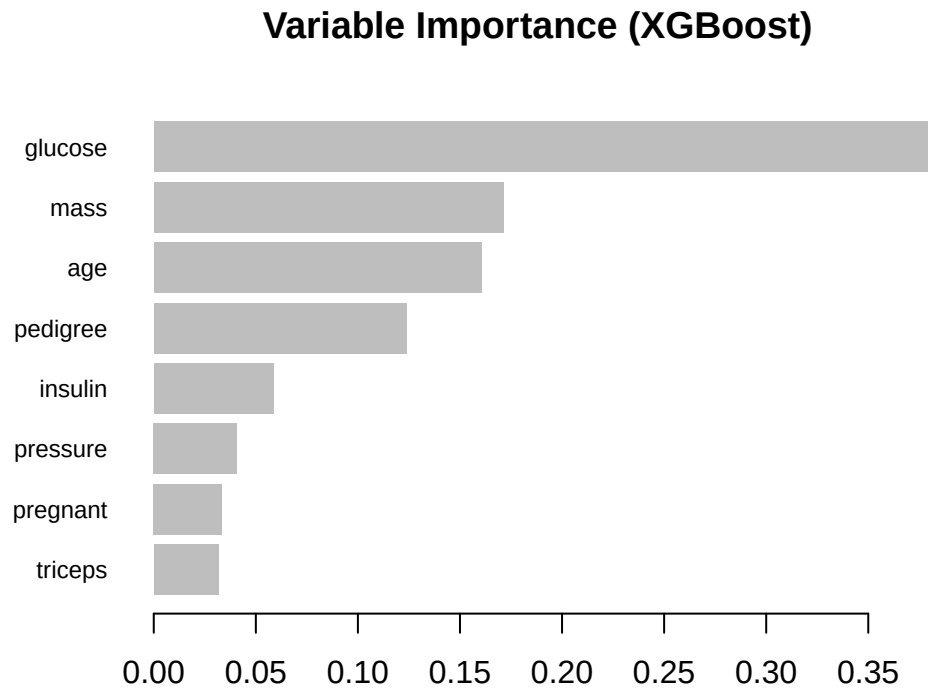
```
xgb_importance <- xgb.importance(
  feature_names = colnames(x_train),
  model = xgb_model
)

print(head(xgb_importance, 8))
```

```
##      Feature      Gain      Cover  Frequency
##      <char>      <num>      <num>      <num>
## 1:  glucose 0.37992146 0.24612658 0.18110236
## 2:    mass 0.17125571 0.19022428 0.18785152
## 3:    age 0.16043052 0.16494617 0.14960630
## 4: pedigree 0.12387697 0.15872439 0.16760405
## 5:  insulin 0.05860213 0.07985235 0.09111361
## 6: pressure 0.04076659 0.06740959 0.09223847
## 7: pregnant 0.03353669 0.05074382 0.06524184
## 8:  triceps 0.03160992 0.04197281 0.06524184
```

```
xgb.plot.importance(
  importance_matrix = xgb_importance,
  top_n = 8,
```

```
main = "Variable Importance (XGBoost)"
)
```



15.1 Logistic regression odds ratios

```
logistic_interpretation_tbl <- broom::tidy(
  log_model,
  conf.int = TRUE,
  exponentiate = TRUE
) %>%
  filter(term != "(Intercept)") %>%
  arrange(desc(abs(log(estimate)))) %>%
  mutate(
    interpretation = case_when(
      estimate > 1 ~ "Higher values associated with higher diabetes odds",
      estimate < 1 ~ "Higher values associated with lower diabetes odds",
      TRUE ~ "No association"
    )
  )

print(logistic_interpretation_tbl)
```

```
## # A tibble: 8 x 8
```

##	term	estimate	std.error	statistic	p.value	conf.low	conf.high	interpretation
##	<chr>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<chr>
## 1	pedig~	2.66	0.332	2.95	3.18e- 3	1.40	5.14	Higher values~
## 2	pregn~	1.11	0.0364	2.93	3.37e- 3	1.04	1.20	Higher values~
## 3	mass	1.11	0.0204	4.94	7.68e- 7	1.06	1.15	Higher values~
## 4	gluco~	1.04	0.00448	8.89	5.89e-19	1.03	1.05	Higher values~
## 5	age	1.02	0.0108	1.53	1.27e- 1	0.995	1.04	Higher values~
## 6	press~	0.987	0.0102	-1.27	2.06e- 1	0.968	1.01	Higher values~
## 7	trice~	0.997	0.0148	-0.233	8.16e- 1	0.968	1.03	Higher values~
## 8	insul~	0.997	0.00133	-2.20	2.81e- 2	0.994	1.000	Higher values~

15.2 Random forest global importance

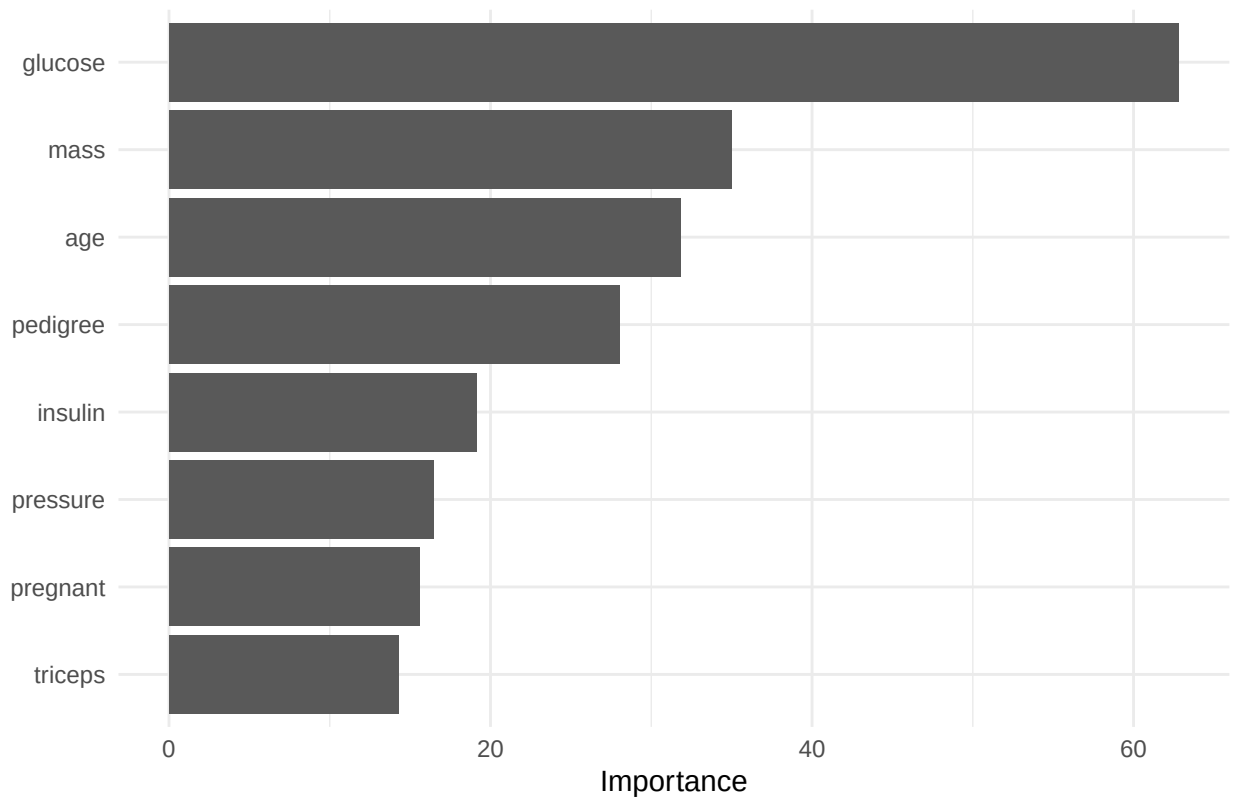
```
rf_importance_tbl <- tibble(
  term = names(rf_model$variable.importance),
  importance = as.numeric(rf_model$variable.importance)
) %>%
  arrange(desc(importance))

print(rf_importance_tbl)
```

```
## # A tibble: 8 x 2
##   term      importance
##   <chr>         <dbl>
## 1 glucose      62.8
## 2 mass         35.0
## 3 age          31.8
## 4 pedigree     28.0
## 5 insulin      19.1
## 6 pressure     16.4
## 7 pregnant     15.6
## 8 triceps      14.3
```

```
rf_importance_tbl %>%
  slice_max(importance, n = 8) %>%
  mutate(term = forcats::fct_reorder(term, importance)) %>%
  ggplot(aes(x = importance, y = term)) +
  geom_col() +
  labs(
    title = "Random Forest Variable Importance",
    x = "Importance",
    y = NULL
  ) +
  theme_minimal()
```

Random Forest Variable Importance



15.3 XGBoost SHAP-style feature contributions

```
# Local feature contributions
xgb_contrib <- predict(
  xgb_model,
  newdata = dtest,
  predcontrib = TRUE
)

xgb_contrib_df <- as_tibble(xgb_contrib)

# Inspect column names (important debug step)
print(colnames(xgb_contrib_df))

## [1] "pregnant"    "glucose"     "pressure"    "triceps"     "insulin"
## [6] "mass"        "pedigree"    "age"         "(Intercept)"

# Remove last column (bias term is always last)
xgb_features_only <- xgb_contrib_df[, -ncol(xgb_contrib_df)]

xgb_shap_long <- xgb_features_only %>%
  pivot_longer(
    cols = everything(),
```

```

    names_to = "term",
    values_to = "contribution"
  )

xgb_shap_summary <- xgb_shap_long %>%
  group_by(term) %>%
  summarise(
    mean_abs_contribution = mean(abs(contribution), na.rm = TRUE),
    mean_contribution = mean(contribution, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  arrange(desc(mean_abs_contribution))

print(xgb_shap_summary)

```

```

## # A tibble: 8 x 3
##   term      mean_abs_contribution mean_contribution
##   <chr>                <dbl>             <dbl>
## 1 glucose                0.944                -0.124
## 2 age                    0.507                -0.0843
## 3 mass                   0.482                -0.133
## 4 pedigree               0.293                 0.00342
## 5 insulin                0.174                -0.00791
## 6 pressure               0.110                 0.0105
## 7 pregnant               0.0990                -0.00482
## 8 triceps                0.0768                -0.00288

```

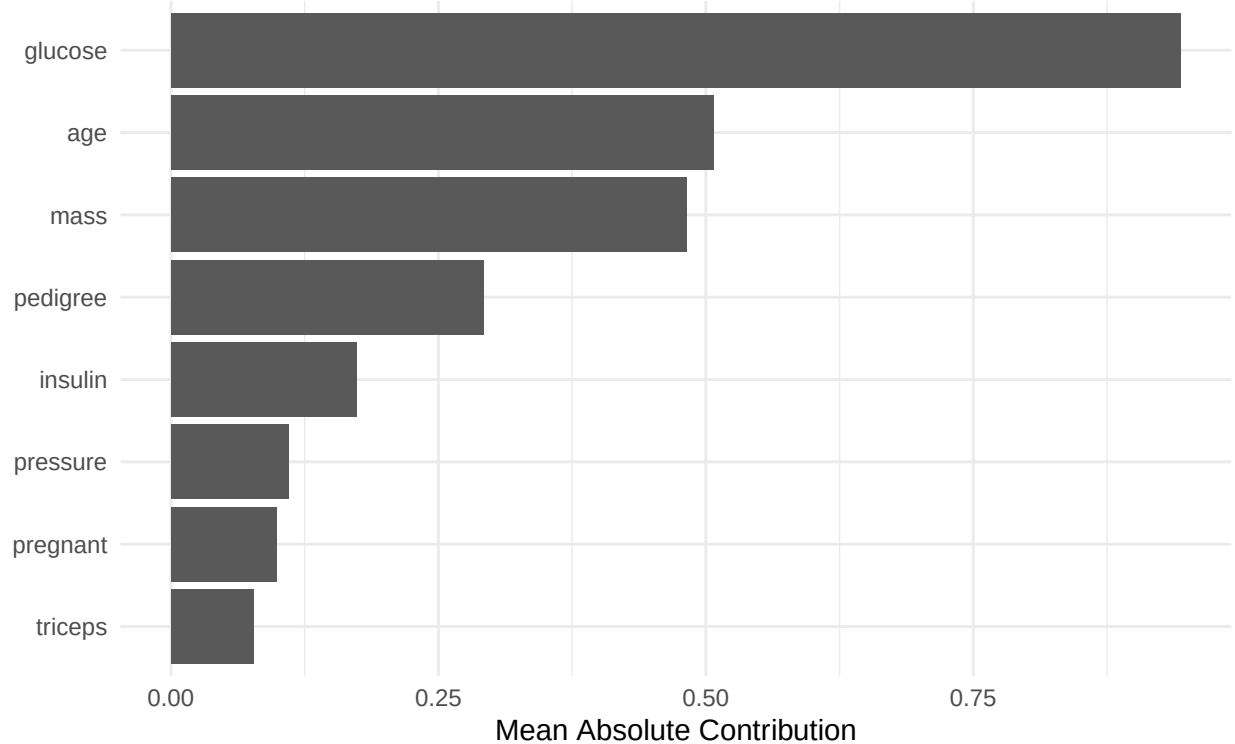
```

xgb_shap_summary %>%
  slice_max(mean_abs_contribution, n = 8) %>%
  mutate(term = forcats::fct_reorder(term, mean_abs_contribution)) %>%
  ggplot(aes(x = mean_abs_contribution, y = term)) +
  geom_col() +
  labs(
    title = "XGBoost SHAP-Style Feature Contributions",
    subtitle = "Mean absolute contribution across test-set predictions",
    x = "Mean Absolute Contribution",
    y = NULL
  ) +
  theme_minimal()

```

XGBoost SHAP-Style Feature Contributions

Mean absolute contribution across test-set predictions



15.4 Side-by-side interpretability summary

```
interpretability_summary <- tibble(  
  model = c(  
    "Logistic Regression",  
    "Random Forest",  
    "XGBoost"  
  ),  
  interpretability_method = c(  
    "Odds ratios from model coefficients",  
    "Global variable importance",  
    "Local SHAP-style feature contributions"  
  ),  
  main_strength = c(  
    "Provides direction and magnitude of association",  
    "Identifies influential predictors in nonlinear model",  
    "Explains feature contributions for individual predictions"  
  ),  
  main_limitation = c(  
    "Assumes linear log-odds relationship unless extended",  
    "Importance is not directly directional or causal",  
    "More complex to explain than odds ratios"  
  )  
)
```

```
print(interpretability_summary)
```

```
## # A tibble: 3 x 4
##   model                interpretability_method    main_strength main_limitation
##   <chr>                <chr>                <chr>          <chr>
## 1 Logistic Regression Odds ratios from model coef~ Provides dir~ Assumes linear~
## 2 Random Forest      Global variable importance Identifies i~ Importance is ~
## 3 XGBoost             Local SHAP-style feature co~ Explains fea~ More complex t~
```

16. Example predicted-risk table

```
risk_table <- test_data %>%
  mutate(
    predicted_prob_logistic = log_prob,
    predicted_class_logistic = log_pred,
    predicted_prob_xgboost = xgb_prob,
    predicted_class_xgboost = xgb_pred
  ) %>%
  arrange(desc(predicted_prob_logistic))

#head(risk_table, 10)

write.csv(risk_table, "predicted_risk_table.csv", row.names = FALSE)
```

17. Save key objects

```
saveRDS(log_model, "logistic_regression_model.rds")
saveRDS(cv_lasso, "lasso_model.rds")
saveRDS(rf_model, "random_forest_model.rds")
saveRDS(xgb_model, "xgboost_model.rds")

#cat("\nProject complete. Files saved to working directory.\n")
```

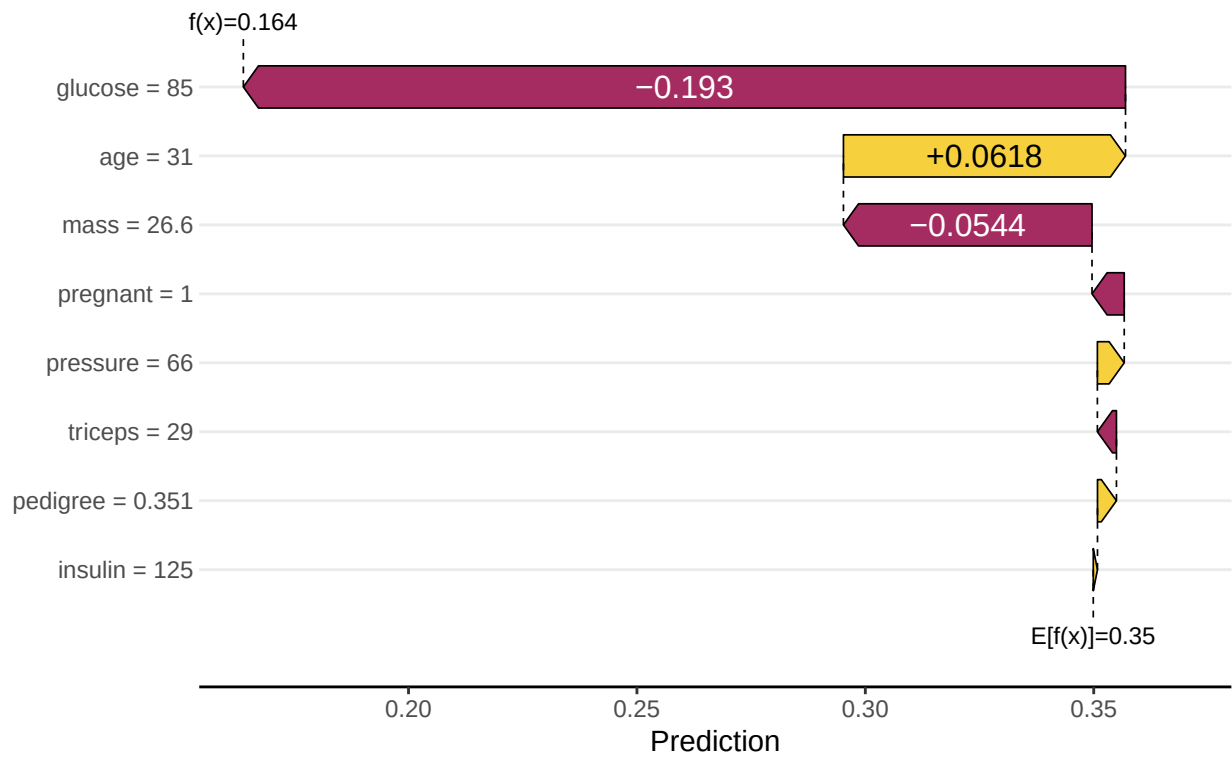
```
pred_fun <- function(m, X) predict(m, X)$predictions[, "pos"]

s <- kernelshap(rf_model,
  X = test_data[, -9],
  bg_X = train_data[, -9],
  pred_fun = pred_fun)
```

```
## |
```

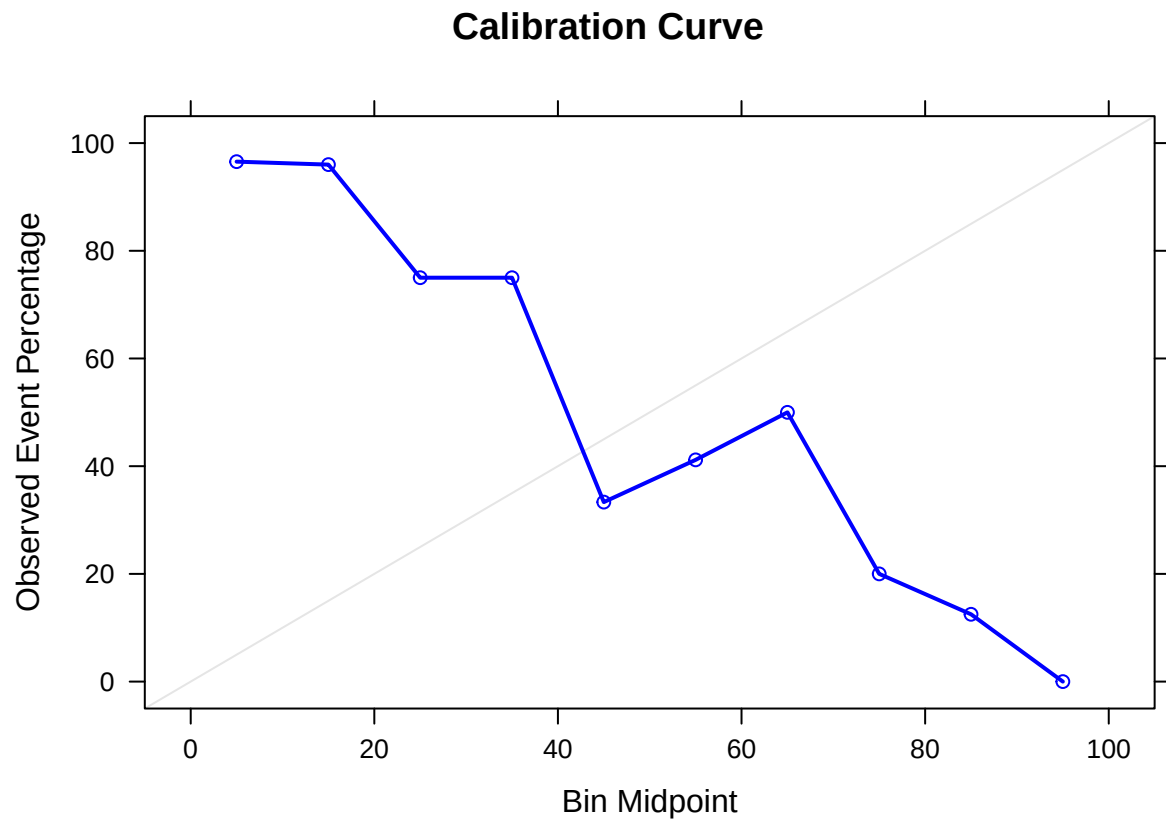
```
sv <- shapviz(s)
sv_waterfall(sv, row_id = 1) +
  ggtitle("SHAP Waterfall")
```


SHAP Waterfall



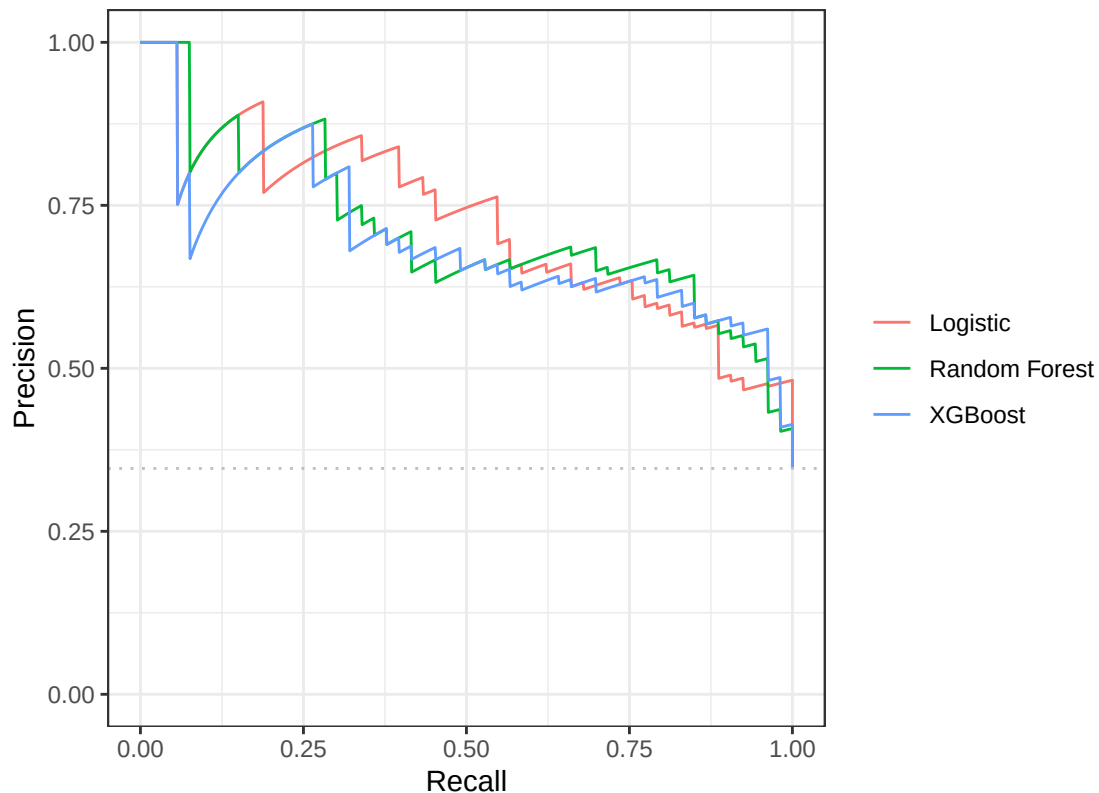
```
calibration_obj <- calibration(diabetes ~ rf_prob, data = test_data, cuts = 10)

plot(calibration_obj,
     main = "Calibration Curve",
     lwd = 2, col = "blue")
```



```
precrec_obj <- mmdata(  
  scores = list(log_prob, rf_prob, xgb_prob),  
  labels = list(y_test, y_test, y_test),  
  modnames = c("Logistic", "Random Forest", "XGBoost")  
)  
  
model_eval <- evalmod(precrec_obj)  
  
autoplot(model_eval, curvetype = "PRC") +  
  ggtitle("Precision-Recall Curve: Performance in Imbalanced Data")
```

Precision–Recall Curve: Performance in Imbalanced Data



```
roc.test(roc_rf, roc_log, method = "delong")
```

```
##
## DeLong's test for two correlated ROC curves
##
## data: roc_rf and roc_log
## Z = 0.13089, p-value = 0.8959
## alternative hypothesis: true difference in AUC is not equal to 0
## 95 percent confidence interval:
## -0.03691191 0.04219493
## sample estimates:
## AUC of roc1 AUC of roc2
## 0.8398113 0.8371698
```

```
# Statistical comparison of AUCs using DeLong tests
```

```
roc_log <- pROC::roc(y_test, log_prob, levels = c("neg", "pos"), direction = "<")
roc_lasso <- pROC::roc(y_test, lasso_prob, levels = c("neg", "pos"), direction = "<")
roc_rf <- pROC::roc(y_test, rf_prob, levels = c("neg", "pos"), direction = "<")
roc_xgb <- pROC::roc(y_test, xgb_prob, levels = c("neg", "pos"), direction = "<")

auc_comparison_tbl <- tibble(
  comparison = c(
    "LASSO vs Logistic",
```

```

  "Random Forest vs Logistic",
  "XGBoost vs Logistic",
  "Random Forest vs XGBoost"
),
auc_model_1 = c(
  as.numeric(pROC::auc(roc_lasso)),
  as.numeric(pROC::auc(roc_rf)),
  as.numeric(pROC::auc(roc_xgb)),
  as.numeric(pROC::auc(roc_rf))
),
auc_model_2 = c(
  as.numeric(pROC::auc(roc_log)),
  as.numeric(pROC::auc(roc_log)),
  as.numeric(pROC::auc(roc_log)),
  as.numeric(pROC::auc(roc_xgb))
),
auc_difference = auc_model_1 - auc_model_2,
p_value = c(
  pROC::roc.test(roc_lasso, roc_log, method = "delong")$p.value,
  pROC::roc.test(roc_rf, roc_log, method = "delong")$p.value,
  pROC::roc.test(roc_xgb, roc_log, method = "delong")$p.value,
  pROC::roc.test(roc_rf, roc_xgb, method = "delong")$p.value
)
) %>%
mutate(
  significant = ifelse(p_value < 0.05, "Yes", "No")
)

#print(auc_comparison_tbl)

write.csv(auc_comparison_tbl, "auc_delong_comparison_results.csv", row.names = FALSE)

```

Main model comparison table

```

knitr::kable(
  results_all,
  digits = 3,
  caption = "Predictive performance across models."
)

```

Table 1: Predictive performance across models.

Model	Accuracy	Sensitivity	Specificity	Precision	Recall	F1	AUC
Logistic Regression	0.758	0.566	0.86	0.682	0.566	0.619	0.837
LASSO Logistic	0.758	0.566	0.86	0.682	0.566	0.619	0.841
Random Forest	0.752	0.566	0.85	0.667	0.566	0.612	0.840
XGBoost	0.745	0.623	0.81	0.635	0.623	0.629	0.836

Table 2: Predictive performance across models.

Model	Accuracy	Sensitivity	Specificity	Precision	Recall	F1	AUC
Logistic Regression	0.758	0.566	0.86	0.682	0.566	0.619	0.837
LASSO Logistic	0.758	0.566	0.86	0.682	0.566	0.619	0.841
Random Forest	0.752	0.566	0.85	0.667	0.566	0.612	0.840
XGBoost	0.745	0.623	0.81	0.635	0.623	0.629	0.836

Table 3: Brier scores for calibration assessment.

model	brier_score
Random Forest	0.154
LASSO Logistic Regression	0.155
Logistic Regression	0.156
XGBoost	0.158

Table 4: Pairwise DeLong tests comparing ROC AUC values.

comparison	auc_model_1	auc_model_2	auc_difference	p_value	significant
LASSO vs Logistic	0.841	0.837	0.004	0.195	No
Random Forest vs Logistic	0.840	0.837	0.003	0.896	No
XGBoost vs Logistic	0.836	0.837	-0.001	0.964	No
Random Forest vs XGBoost	0.840	0.836	0.004	0.706	No

Table 5: Odds ratios from logistic regression.

term	estimate	std.error	statistic	p.value	conf.low	conf.high
pedigree	2.662	0.332	2.949	0.003	1.398	5.142
pregnant	1.113	0.036	2.932	0.003	1.037	1.196
mass	1.106	0.020	4.943	0.000	1.064	1.152
glucose	1.041	0.004	8.894	0.000	1.032	1.050
age	1.017	0.011	1.526	0.127	0.995	1.038
insulin	0.997	0.001	-2.196	0.028	0.994	1.000
triceps	0.997	0.015	-0.233	0.816	0.968	1.026
pressure	0.987	0.010	-1.265	0.206	0.968	1.007
(Intercept)	0.000	0.899	-9.950	0.000	0.000	0.001

Table 6: XGBoost SHAP-style feature contributions.

term	mean_abs_contribution	mean_contribution
glucose	0.944	-0.124
age	0.507	-0.084
mass	0.482	-0.133
pedigree	0.293	0.003
insulin	0.174	-0.008
pressure	0.110	0.010

term	mean_abs_contribution	mean_contribution
pregnant	0.099	-0.005
triceps	0.077	-0.003

Conclusion: Logistic regression remains a strong and competitive method for diabetes prediction. In this study, it achieved predictive performance comparable to modern machine learning models while offering superior interpretability. These findings support the continued use of logistic regression as a baseline model in biomedical informatics, particularly in settings where transparency and clinical trust are essential.